

Net interview questions for freshers

1.What is the .NET framework?.

Ans:- .NET is a platform introduced by Microsoft to develop, host, deploy, maintain and execute the application. In other words, it caters to the entire development life cycle of the application, making developer life more comfortable. The .NET framework supports an object-oriented way to build various applications. These applications could be database-driven client-server applications, desktop or windows applications, web applications like Online banking portals, Online shopping websites, web applications, and web services.

Using .Net, the applications are developed using languages such as C#, VB.NET, etc. This is one of the most important.net interview questions asked by an interviewer.

2.What are the different versions of the .NET framework available?

Version	.NET Framework	Visual Studio	
C# 1.0	.NET Framework 1.0/1.1	In Visual Studio .NET 2002	
C# 2.0	.NET Framework 2.0	In Visual Studio .NET 2005	
C# 3.0	.NET Framework 3.0/3.5	In Visual Studio .NET 2008	
C# 4.0	.NET Framework 4.0	In Visual Studio .NET 2010	
C# 5.0	.NET Framework 4.5	In Visual Studio .NET 2012/2013	
C# 6.0	.NET Framework 4.6	In Visual Studio .NET 2013/2015	
C# 7.0	.NET CORE Frameworks	In Visual Studio .NET 2017	

3.What are the different components of the .NET framework?

Ans:- Below following are the .NET components:

- Common Language Specification
- Common Language Runtime
- Framework Class Library

- WebForms
- Windows Forms
- Framework Base Classes
- Application Domain

4.What is Namespace?

Ans:- It's a logical group of classes. For example, **System.Web.UI** namespace is a logical group of UI-related classes containing features such as text boxes, list control, etc.

Using keyword is used for using the namespace.

5.What is code access security (CAS)?

Ans:- Code access security (CAS) is a part of the .NET framework security model that stops unauthorized access of resources and operations and restricts the code to perform particular tasks.

6 .How can you turn on CAS?

Ans:- To turn on CAS, run the below command in the command prompt:

caspol -security on

7.How can you turn off CAS?

Ans:- To turn off CAS, run the below command in the command prompt:

caspol -security off

8.Which is the root namespace for fundamental types in .NET Framework?

Ans:- System.Object

9.What is CLS?

Ans:- CLS stands for Common Language Specification. .Net language support both CLS and CTS. The CLS includes a subset of the Common Type System (CTS). CLS defines rules for declaring, using, and managing types by the runtime environment. The CTS defines the rules concerning data types. Below are some examples of CLS rules

- Use of **new** keyword to instantiate a class
- The **array** should be zero-based.
- Third-party should be object-oriented.

A component of a .NET application created in one language can be used by other languages if both satisfy the CLS rules. Such components are referred to as **CLS-compliant components**.

10.What is CTS?

Ans:- CTS stands for Common Type System. The CTS defines the rules concerning data types.

It ensures type safety and cross interoperability. CTS supports two general categories of types which are derived from the base type called **System. Object**

- **Value types**
- **Reference types**

Variables used in managed code of the .NET application are defined using CTS. It is a superset of data types used in managed code by all .NET compliant languages such as C# and VB.NET.

11.What is CLR?

Ans:- CLR stands for Common Language Runtime. CLR is an execution for .NET applications. Managed code is targeted to CLR, which internally compiles Microsoft Intermediate Language (MSIL) code into platform-dependent instructions.

CLR provides the following managed code execution services.

- **Secure execution of code**

CLR converts the MSIL code into the native code.

- **Automatic Memory Management**

Automatic Memory Management for objects created while executing MSIL code by CLR is managed by Garbage Collector.

- **Interoperability**

Objects of classes written in one language can interoperate with objects of classes in other languages.

- **Exception Handling**

CLR supports a standard exception-handling mechanism that works across all languages, allowing every program to use a common error-handling mechanism.

12.What is Managed Code?

Ans:- Assemblies consist of managed code that is targeted to CLR for execution. Managed code is generated by compilers of VB.NET, C# by default. Due to managed code .NET framework supports Language Interoperability. Multiple programming languages can be used to build a single application. Managed code is a code managed by CLR. To execute the managed code .NET framework is needed. CLR handles memory management via garbage collection.

13.What is Unmanaged code?

Unmanaged code is a code that is not managed by CLR. To execute the unmanaged code .NET framework is not needed; it is independent of the .NET framework. For execution and compilation, unmanaged code has its own runtime environment.

14.What is a Garbage collector?

A Garbage collector is a feature of CLR. This performs the periodic cleanup operation on unused managed objects to check if any unused or unreferenced objects exist whose memory can be reclaimed. It doesn't perform the cleanup operation on unmanaged objects. This avoids memory leakage and provides efficient memory management.

15.What are generations in Garbage collector(GC)?

This is one of the common .net interview questions asked in an interview.

Generations denote the age of objects. Generations are created to improvise the GC performance.

There are three types of generations **Gen 0**, **Gen 1**, and **Gen 2**.

1. **Gen 0:** While creating an application, new objects are created; they are marked as Gen 0. GC spends will spend more time on Gen 0 instead of Gen 1 and Gen 2.
2. **Gen 1:** When the garbage collector performs the cleanup operation on Gen 0 and found an active object whose reference exists, those objects are moved to Gen 1.
3. **Gen 2:** When the garbage collector performs the same periodic cleanup operation on Gen 1 and found an active object whose reference exists, those objects are moved to Gen 2.

16.Garbage collector cleans managed code; how do we clean unmanaged code?

The garbage collector performs the cleanup operation on managed code to reclaim memory. For unmanaged code, we can perform the cleanup using destructor / finalize.

17.How can we force the garbage collectors to run?

We can force the garbage collectors to run by using **"System.GC.Collect()"**.

18.What is the difference between "dispose" and "finalize"?

Dispose: Dispose uses the interface **"IDisposable"**. The user program needs to be called a function to clean up the unmanaged code.

Finalize: finalize is a destructor, and it is internally called by the garbage collector to clean up the managed code and can't be called from code.

19.What is Assembly Loader?

Assembly loader is one of the components of CLR. It first located the assembly, which contains the **Main()** method and then loads it into memory. It also locates and loads other classes from FCL, which are referenced in the code.

20.What is JIT?

JIT stands for Just-In-Time Compiler(JIT). The standard JIT compiler runs on demand. When a method is invoked, the JIT compiler analyzes the MSIL code and produces highly efficient machine code. A small module is injected per method, which contains the information on whether the method is already compiled. JIT compiler takes this information and compiles only the code which is not yet converted to native code. As the .NET applications run, they tend to become faster and faster as the already-compiled code is reused.

21.What are the different types of JIT compilers?

There are three types of JIT compilers:

- **Standard JIT:** It is used in the development of applications.
- **Pre JIT:** if we want to avoid the cost of JIT compilation at runtime, we can use a special tool called **ngen.exe**, which compiles IL(Intermediate Language) during installation and setup time. Using the **ngen** tool, we can JIT-compile the code once and cache it on the machine so that we can avoid JIT compilation at runtime.
- **Econo JIT:** It is used for mobile applications.

22.What is Application Domain?

Every .NET application is executed in a dedicated worker process on Windows Operating System. To execute managed code of the .NET application, CLR loads managed code in the application context of the process called Application Domain. Here is what they do:

- Multiple application domains can run in a single process
- Application domains are typically created by runtime hosts responsible for bootstrapping the common language runtime before an application is run.

23.What is Assembly?

Ans:- Assembly is a logical unit of deployment and version control of the .NET application. As the assembly is a collection of types and resources that are built to work together. The .NET Assemblies could be in the form of an application(EXE) or a class library (DLL). CLR provides the execution environment for the assemblies.

24.What are the components of Assembly?

Ans:- Following are the main components of an Assembly:

- **Manifest**

It contains the assembly information such as assembly's version, security, identity, and references to other assemblies.

- **Type Metadata**

Metadata provides information about all the types defined in the assembly, such as classes, interfaces, properties, and so on.

- **MSIL code (Microsoft Intermediate Language Code)**

This is the Microsoft Intermediate Language Code. It is a CPU-independent set of instructions that can be efficiently converted into native code. Before the code is executed, the JIT compiler of CLR converts it to CPU-specific native code. When MSIL is produced, the compiler also creates metadata for that.

- **Resources**

Resources are like .bmp or .jpg files. General allocation is one resource file per application.

25.Where is the version information stored in an assembly?

Ans:- It is inside the manifest.

26.What is MSIL code?

Ans:- This is the Microsoft Intermediate Language Code. It is a CPU-independent set of instructions that can be efficiently converted into native code. Before the code is executed, the JIT compiler of CLR converts it to CPU-specific native code. When MSIL is produced, the compiler also creates metadata for that. Combined with metadata and CTS, MSIL allows for true cross-language Interoperability.

27.What are the features of Assembly?

Ans:- Assembly provides several features like the once mentioned below:

- Assembly can consist of a file or a collection of different modules.
- Assemblies are the smallest functional unit of deployment.
- They are deployed on any platform where the .NET framework is installed.
- Execution of Assemblies is controlled using Code Access Security.

28.What is ILDASM?

Ans:- Content of Assembly could be viewed using a tool called ILDASM.exe.

29.What are the different types of Assembly?

Ans:- .NET application is made up of one or more assemblies. Assemblies could be of two types:

1. Private Assembly
2. Shared Assembly

30.What is a Private Assembly?

Ans:- Private assemblies are created when the functionality of a component or .dll is specifically associated with an application.

Private assemblies are created in the Application folder, which is the **bin** folder of the .NET application.

31.What is Shared Assembly?

Ans:- Class libraries, which are shared between applications, are called Shared Assemblies. For Example, **System.winform.dll** provides GUI-based support for all .NET based windows applications.

32.What is GAC?

Ans:- This is one of the common .net interview questions asked in an interview.

GAC stands for Global Assembly Cache(GAC).GAS is a shared assembly area in the .NET platform where most assemblies are installed. Framework Class Library(FCL) assemblies are also installed in GAC. These assemblies are shared assemblies. All shared assemblies have to be strong-named assemblies. The string name is associated with assembly using .NET Tool **sn.exe**.

33.What is C#?

Ans:- C# language is managed object-oriented programing language. C# language uses the simplicity of Java and the power of C++ language. C# language is a type-safe object-oriented language. It supports all the major and minor pillars of object-oriented programming. It is case sensitive and has its own set of keywords to represent the program. C# source code syntax is similar to C, C++, and Java. C# has access to Framework Class Library (FCL) of .NET for application development. C# source code can access both unmanaged and managed code in the .NET application. A legacy application written in DLL or using COM can be accessed in a .NET application due to the Interoperability concept. NET.

34.How .NET application executes?

Ans:- .NET source code is compiled twice before execution. Below are the steps of compilation:

1. Source code (C# or VB.NET) is compiled into MSIL (Microsoft Intermediate Language) with the help of the language-specific compiler. For example, CSC compiler for C# and VBC compiler for VB.NET.
2. C# complier compiles source code into MSIL code.
3. MSIL code is stored in a file with the extension either .dll or .exe. It is a logical unit of deployment called assembly.

4. The MSIL code is then compiled by the Just-in-Time(JIT) compiler, which is part of CLR it compiles it into native executable code.
5. This executable code is then processed by the machine's processor.

35.What is Value type?

Ans:- Values types directly contain the data. Actual data is always stored on the stack.

36.What is a Reference type?

Ans:- Reference types store a reference to the memory address on the stack, but the actual data is stored on the heap.

37.Difference between Value type and Reference type?

Ans:-

Value Type	Reference Type
The variable holds the actual value.	The variable holds Memory location.
The default value is 0	The default value is null
Actual data is stored on the stack.	Actual data is stored on the heap.
It always has value.	May be null
e.g., int, float, Enum, struct	e.g., object, string, classes, interface, arrays

38.What is boxing?

Ans:- Boxing is converting a value type to a reference type. It is also called an Implicit conversion.

E.g.:

int rank=10;

Object sp = rank;

39.What is unboxing?

Ans:- Converting reference type to value type. It is also called an explicit conversion.

E.g.:

Object sp=100;

int speed=(int)sp;

40.What is Enum?

Ans:- Enum is the Value type that consists of a set of named integer constants. Each member starts from 0 by default and is incremented by 1 for each next member. Enums are not needed to follow a sequential ordering.

41.What is a struct?

Ans:- The structure is a value type. It is used while developing mathematical or geometrical applications. It helps create new value-type objects similar to the built-in type (int, float, bool, and so on).

If we have a class referencing value type, where is the value type get stored?

The value type gets stored inside a Heap.

42.What is the difference between “constant” and “readonly” variables in C#?

Ans:- A **Constant** keyword is used for making variables constant. We can't modify the value of the constant.

Constant variables are evaluated at compile-time. It supports value type variables.

The readonly keyword is used for making the variable read-only. Readonly variables are evaluated at runtime. It supports reference type variables.

43.In how many ways can we pass parameters to the method?

Ans:- Parameters can be passed to the methods in two ways: **reference** and **output**

ref(reference) parameters:

ref parameters should be initialized. When parameters are passed using **ref**, the method refers to the same variable that was passed into the method. Any changes made to the parameters in the method will be reflected in the variable when control is passed back to the calling method.

out(output) parameters:

out parameters need not be initialized. But they have to assign some value before the control returns to the calling method. Using out parameters multiple values can be returned.

44.What is Array?

An array is a set of similar elements stored in a contiguous memory location. It is a reference type in the .NET framework. The base index is always zero. The size of the array is not part of its type as it is in the C

language. This allows to declare an array and assign an array of int objects to it, regardless of the array's length.

45.What are the different types of arrays?

Ans:- Arrays are of two types:

1. Single dimensional arrays
2. Multi-dimensional arrays.

46.What are Multi-dimensional arrays?

Ans:- Multi-dimensional arrays are an array which can have more than one dimension.

e.g.: the following created a two-dimensional array of three rows and two columns:

```
int [ , ] numbers= new int[3,2] {{1,2},{3,4},{5,6}};
```

47.What is a Jagged Array?

Ans:- A jagged array is an array whose elements are also arrays. The elements of a jagged array can be of different dimensions and sizes. A jagged array is sometimes called an "array-of-arrays".

e.g.:

```
int [ ][ ] jaggedArray = new int[3][ ];
```

48.What is Object?

Ans:- An object is a real-world entity. An entity that has a well-defined structure and behaviour. An object is a reference type i.e nothing but the instance of a class. This is used to access the methods and properties of a class. Below are the characteristics of an object:

- State
- Behavior
- Identity
- Responsibility

Now that we have seen .net interview questions and answers for freshers, that has been asked in many companies Let's see the list of top . Net interview questions for experienced.

49.What is a delegate in .NET?

Ans:

A delegate in .NET is a function pointer like C or C++. A delegate allows the user to encapsulate the reference and can then be passed in a program, which will call the referenced method.

50.Difference between interface and abstract class in .NET?

Ans:

Interface	Abstract Class
An interface declares behaviour that implementing classes should have.	An abstract class provides a partial implementation for functionality by the inheriting entities.
An interface can declare only properties, methods and events.	An abstract class declares fields too.

51.What is the difference between a stack and a heap?

Ans:

Stack	Heap
Value type	Reference type
A stack is responsible for keeping track of all the executing threads and their location.	The heap is responsible for keeping track of the precise data.

52.What is the difference between function and stored procedure?

Ans:

Function	Stored Procedure
Can return a single value	Always used to perform a specific task
Stored Procedure can have the input parameter	Stored Procedure can have both input and output parameters
A stored procedure cannot be called from a function	A function can be called from a procedure

.Net Interview questions for experienced

1.What is Class?

Class is a user-defined type; it is a reference type. It is a blueprint of the object. A class is an abstract data type that maps real-world entities through data members(constants) and member function(methods, properties, events, and so on).

2.What is the Main() method in C#?

The Main() method is the entry point of a C# program.

3.Is C# type safe Object-oriented programming language?

Yes

4.What is the default access modifier for a class?

internal is the default access modified for a class.

5.What is the default access modifier for a class member?

private is the default access modifier for a class member.

6.What is Object-Oriented Programming?

Object-Oriented Programming is composed of discrete objects interacting with each other to give rise to the system's overall behaviour. These objects need to interact with each other during the execution of a system. Consider an example of a C# training module where students, faculty, training course, classroom, whiteboard, desktop, and final tests are different objects with their attributes. These objects need to interact with each other to result in the successful completion of the module.

7.What are the main pillars of Object-Oriented Programming?

OOP defines the following major pillars:

- Abstraction
- Encapsulation
- Inheritance
- Polymorphism

8.What is Abstraction?

Abstraction is a pillar of object-oriented programming language. It is a mechanism to defines the required information of an object and hides the irrelevant details. The essential characteristics(properties and functions) of an object distinguish it from all other kinds of objects.

Abstraction is implemented with the help of **abstract classes** in C#.

E.g., A customer is using an ATM for withdrawing money. The customer only knows that how to use the ATM by inserting an ATM card, then pin code of the ATM card and later entering the amount which he/she wish to withdraw and last collecting the money. But they don't know the internal process of ATM-like how the money card is read and how the money gets withdrawn from the ATM etc.

Here customers can only know how to operate the ATM that is called abstraction.

9.What is Encapsulation?

Encapsulation is a pillar of object-oriented programming language. This deals with data hiding and providing the interface to the end-user of the object. This is implemented by using **access modifiers** we can hide the required details and can display only mandatory **methods** and **properties** with the reference of an object.

E.g.: A car is the perfect example of abstraction. A car contains a steering wheel, seatbelt, gear, door, and a clutch. To use the car, we just need to know how to operate it, we don't need to know which internal parts are used, and how all internal parts are working.

10.What is Inheritance?

Inheritance is a pillar of object-oriented programming language. Inheritance defined the relationship between classes, which share common behaviour and structure. Inheritance is shown by the relationship between classes- as a base class and a derived class. Base class defines common structure and behaviour which have been reused by other classes through the hierarchy. Derived classes provide an extension of structure and behaviour to base classes.

E.g.: Consider an example of a bank system. A banking account is a term in the banking domain. A saving account is a banking account indeed and hence it can form a hierarchical relationship. It will inherit the features of a banking account (reusability) and add its own features. A new category current account can be added to the banking account hierarchy making the design extensible.

Also Read: [Top 50+Nodejs Interview Question and Answer 2021](#)

11.What is Polymorphism?

Poly means **many** and **morph** means **forms**.

Polymorphism is a feature in which a single name represents many unlike objects related to a common parent object. This single object can respond to a common set of operations. Polymorphism exists when features of Inheritance and dynamic binding interact with each other.

E.g., Suppose a Shape is a generic object which represents Circle, Rectangle, and Triangle. A draw is a common operation that can be performed on Shape; only the implementation to draw will differ with circle or rectangle or triangle.

12.What are the different types of Access modifiers in C#?

The C# language support five access modifiers of members is given using public, private, protected, internal, and protected internal modifiers.

Public:

It is accessible everywhere. There are no restrictions on accessing public members.

Private:

It is accessible only within the body of the class in which they are declared. Compiler time error occurs if accessed outside the body of the class.

Protected:

It is accessible within the class in which it is declared and within derived classes that declared this member.

Internal:

It is accessible only within the current project.

Protected internal:

It is accessible within the current project and derived classes.

13.What are nested classes?

Classes inside the classes are called nested classes.

14an we return multiple values from a function in C#?

Yes, using our (output) parameter, we can return multiple values from a function in C#.

15.Which class acts as a base class for all the data types in .net?

Object class is the base class of all the data types in .net. As in .NET programming languages, all types are inherited from the **Object** class.

16.What are params? Can we create a function in C# that can accept a varying number of arguments?

Yes, by using the **params** keyword. C# supports the use of parameter arrays. **Params** keyword defines a method that can take a variable number of arguments. No additional parameters are accepted after the **params** keyword in a method is declared. Only one **params** keyword is permitted in a method declaration.

17.What is Constructor?

An object is created on the managed heap using a **new** operator, which calls the constructor of the class. Constructor is used for initializing the object members. More than one constructor could be written within Class.

18.What are the features of the Constructor?

This is one of the common .net interview questions asked in an interview.

- In C#, constructors have the same name as that class.
- It can be overloaded.
- It doesn't have any return type, not even **void**.
- Constructors need to be public; otherwise, if private, object initialization is not allowed.
- If no constructor is specified for a class, the compiler generates a default constructor.
- The constructor can be static, and it called only once before the instance of the class is created. It must be without any parameters and with no access modifier(private, public).

19.What is Destructor?

Destructor is a method of the class used to De-initialize resources allocated during the initiation of an object. In C#, the destructor is called Garbage Collector, which is a component of CLR to reuse the object's memory when it is not required.

20.What are the features of Destructor?

- In C#, Destructor has the same name as that class.
- It has no return type and doesn't take any parameters.
- A ~ character preceding the name.
- Cannot be overloaded.

21.What are the different types of Constructors?

In C#, the following are the different types of Constructors:

- Default Constructor
- Parameterized Constructor
- Private Constructor
- Copy Constructor
- Static Constructor

22.What are properties?

Properties are known as smart fields. Fields are data members of the class. Properties are extensions to fields and can be accessed using the same syntax.

They have two accessors:

- get- retrieve data member values
- set- set data member values

If only **get** is specified, then the property becomes a read-only property. If the only **set** is specified, then the property becomes a write-only property. If both are specified, then the property is read/write property.

23.What is the Static method?

The static method can access static members only. They are invoked using the class name. A static method doesn't allow the creation of an instance of the class.

24.What is a Static member?

Static member belongs to the type class itself rather than to a specific object. Therefore, only a single copy of static members exists per class. It is shared by all objects of the class.

25.What is the Non-Static method?

Non-Static methods are also known as instance methods as they are always called with the object of that class. In other words, the Non-Static method is invoked using object reference.

26.What are Exceptions?

This is one of the common .net interview questions asked in an interview.

Exceptions are errors that occur during the execution of a program; they are also called runtime errors. Most of the time, they can be detected and handled in code. .NET framework terminates the program execution for runtime error.

e.g., validating user input, divide by zero, and stack overflow.

The **System.** Exception class provides several methods and properties for obtaining information on what went wrong. When exceptions occur, they are said to throw what is actually thrown as an object that is derived from the System. **Exception** class.

27.What are the different types of Exceptions?

There are two types of Exceptions:

- System-level Exceptions
- Application-level Exceptions

28.What are System-level and application-level exceptions?

1. **System-level Exceptions:**
2. .NET framework thrown an exception are called System-level exceptions. These exception classes are derived from **SystemException** class.
3. e.g., Index out of range exception. Array out of bound error.
4. **Application-level Exceptions:**
5. it is custom exceptions created for the application and are thrown by the user program. Their exceptions classes are derived from **ApplicationException** class.

ApplicationException is thrown by the user program, not by the common language runtime.

30.What is Exception Handling? And how to handle it?

This is one of the common .net interview questions asked in an interview.

The process to detect and handle runtime errors is called **Exception Handling**.

C# provides **try-catch** blocks to handle these runtime errors effectively.

Code that could throw an exception is put in the **try** block, and an exception-handling code goes in the catch block.

Irrespective of whether the exception is thrown, **finally** block is always get executed. It is the block where one can write code like closing the file or connection or releasing resources.

31.Can try block contains multiple catch blocks?

Yes, there can be multiple **catch** blocks for one **try** block.

32.What is a partial class?

A class can be spread across multiple source files using the keyword **partial**. All source files for the class definition are compiled as one file with all class members.

33.Does C# support multiple inheritance?

No, C# doesn't support multiple inheritances.

34.What are early binding and late binding?

This is one of the common .net interview questions asked in an interview.

Early binding:

In early binding methods and properties are detected and checked during compile-time. Early binding is also called static binding. Example of early binding is Method overloading and Operator overloading.

Late binding:

Late binding is the opposite of early binding. In the late binding method and properties are detected and checking during runtime. Late binding is also called dynamic binding. An example of dynamic binding is Method overriding.

35.What are the different types of polymorphism?

Polymorphism can be achieved using inheritance and virtual keyword

There are two types of polymorphism:

- Static or compile time polymorphism
- Dynamic or runtime polymorphism

Static polymorphism is implemented using **Function or Operator overloading**, while dynamic polymorphism is implemented using **method overriding**.

36.What is Method Overloading?

Methods with the same name but different signatures are said to be overloaded methods.

The signature of a method is the datatypes of the parameters, the number of parameters, and the sequence in which they are passed to the function.

37.What is Operator overloading?

The process of defining the meaning of an operator relative to a class is called operator overloading. C# provides type extensibility power through operator overloading. Operator overloading is closely related to method overloading. To define the meaning of the operator, we have to define a function called an **operator** function. This function is defined as the **operator** keyword, and it defines the action of the operator relative to its class.

The operator function must be **public** and **static**.

38.What is method overriding?

In method overriding, methods have the same names, same signatures, same return types but in different classes in the hierarchy. C# use **virtual** and **override** keywords to define method overriding. Method overriding enables an object to resolve method invocation at runtime. It is also called **late binding**.

39.What is Shadowing?

Shadowing is hiding the base class member in the derived class by using the keyword **new**.

40.What is a Sealed class?

A sealed class cannot be inherited. The class can be stopped from further inheriting using the **Sealed** class.

41.What is an abstract class?

The **abstract** keyword enables the creation of classes and class members for inheritance. Abstract class must contain at least one abstract method. other methods can be abstract or non-abstract. Abstract class doesn't allow the creation of an object of the class.

42.What is an interface?

The Interface is a reference type. It cannot contain data members. It contains the only declaration of methods, properties, indexers, and events. But the implementation part can be done in the class that implements the interface. The Interface is public by default.

43.Can we create an object of Abstract class or an interface?

No, we cannot create

44.What are the advantages of inheritance?

Reusability and extensibility.

45.What are Nullable types?

Value type doesn't contain a null value. So, it becomes difficult to assign a null to value types like int, float, bool, etc.; by using nullable types, we can assign null to value type. To create a nullable type, we need to put '?' before the data type like below:

```
int? age=null;
```

46.What are collections?

A collection is a set of similar types of elements that are grouped. **System. Collections** namespace contains interfaces and classes that defined various collections of objects such as lists, queues, hash tables, dictionaries, and so on.

47.What are the different types of collections in .NET?

There are five different types of collections in .NET:

Arrays, lists, queues, stacks, and hash tables.

48.What is a generic collection?

The generic collection is defined in **System.Collection.Generic** namespace. Generic helps to separate logic from data type. It provides type safety. Generic code is once written, can be reused by instantiating the class of the same type.

Generic examples are: List<T>, Stack<T>, Queue<T>, Dictionary<T>. (where T can be any data type.)

49.What is the difference between Array and ArrayList?

Arrays have a fixed size, whereas ArrayList size is resizable.

Arrays can store any type of data but only one type when it is strongly typed, whereas ArrayList can store any data type.

50.How does the .NET framework work?

This is one of the common .net interview questions asked in an interview.

- .NET framework-based applications are written in supportive languages like C#, F#, or Visual basic.
- Compiled code is stored in the form of an assembly file that has a .dll or .exe file extension.
- When the .NET application runs, complied code takes the assembly file into machine code with the help of the Just In Time(JIT) compiler.
Now, this machine code can execute on the specific architecture of the computer it is running on.

51.What is the difference between managed and unmanaged code?

Managed code	Unmanaged code
Managed code is managed by CLR	Any code that is not managed by CLR
.NET framework is necessary to execute managed code	Independent of .NET framework
CLR manages memory management through garbage collection	Own runtime environment for compilation and execution

52.What do you know about boxing and unboxing?

Ans:

Boxing	Unboxing
Implicit	Explicit
Converts from value type to the type object	Extracting the value type from the object

53.Differentiate between constants and read-only variables.

Ans:

Constants	Read-only Variables
Evaluated at compile time.	Evaluated at run-time.
Support only value-type variables.	Support reference type variables.
Cannot be initialized at the time of declaration.	Can be initialized at the time of declaration.

54.What is the difference between custom and user control?

Ans:

Custom Control	User Control
Derives from control	Derives from UserControl
Can be added to the toolbox	Cannot be added to the toolbox
Loosely coupled control	Tightly coupled control
Dynamic Layout	Static Layout
Defines a single control	Defines a set of con

55.Difference between localization and globalization?

Ans:

Localization	Globalization
It means changing the already globalized application to support a specific language.	Globalization is the process of developing applications to support multiple languages.
Localization is used to localize the application content.	Can also be converted to support multiple languages.

56.What is a delegate in .NET?

Ans:

A delegate in .NET is a function pointer like C or C++. A delegate allows the user to encapsulate the reference and can then be passed in a program, which will call the referenced method.

57.Difference between interface and abstract class in .NET?

Ans:

Interface	Abstract Class
An interface declares behaviour that implementing classes should have.	An abstract class provides a partial implementation for functionality by the inheriting entities.
An interface can declare only properties, methods and events.	An abstract class declares fields too.

58.What is the difference between a stack and a heap?

Ans:

Stack	Heap
Value type	Reference type
A stack is responsible for keeping track of all the executing threads and their location.	The heap is responsible for keeping track of the precise data.

59.What is the difference between function and stored procedure?

Ans:

Function	Stored Procedure
Can return a single value	Always used to perform a specific task
Stored Procedure can have the input parameter	Stored Procedure can have both input and output parameters
A stored procedure cannot be called from a function	A function can be called from a procedure